



**FACULDADE CESAR SCHOOL
CURSO DE GRADUAÇÃO EM 2026**

**TEORIA DOS GRAFOS
RELATÓRIO FINAL DO PROJETO FINAL**

**Julia Torres de Barros
Maria Clara de Souza Almeida Neves
Maria Cláudia Rodrigues Corrêa de Oliveira Andrade
Vinícius Bernardo da Silva**

**RECIFE
05/2026**

Maria Clara de Souza Almeida Neves
Vinícius Bernardo da Silva
Julia Torres de Barros
Maria Cláudia Rodrigues Corrêa de Oliveira Andrade

RELATÓRIO FINAL DO PROJETO FINAL

Relatório final do projeto final apresentado para o cumprimento de Teoria dos Grafos do Curso de Graduação em Ciência da Computação da Faculdade CESAR School, realizado no período de 26/03/2026 a 11/06/2026, sob a supervisão de Anderson Gomes e Laura Alves Pacifico da Silva, professora de Teoria dos Grafos do curso de Ciência da Computação da Faculdade CESAR School.

RECIFE
06/2026

Resumo

Este trabalho apresenta o desenvolvimento do projeto final da disciplina de Teoria dos Grafos do curso de Ciência da Computação da CESAR School, cujo objetivo consistiu na modelagem de conjuntos de dados reais como grafos e na aplicação prática dos algoritmos de BFS, DFS, Dijkstra e Bellman-Ford, com posterior análise dos resultados obtidos.

O projeto foi estruturado em duas partes distintas. Na Parte 1, foi modelada a malha aérea brasileira a partir de um dataset de 20 aeroportos, representados como vértices identificados pelo código IATA. As arestas foram definidas manualmente com base na importância operacional dos aeroportos, adotando-se um modelo de pesos híbrido que combina penalidades por mudança de região geográfica e pela ausência de aeroportos hub, resultando em um grafo não direcionado com 73 arestas e densidade aproximada de 0,384. A aplicação dos algoritmos BFS e DFS confirmou a conectividade do grafo, enquanto o Dijkstra foi utilizado para encontrar caminhos de custo mínimo entre pares inter-regionais, revelando que conexões entre hubs reduzem significativamente o custo efetivo das rotas. A análise de métricas intra e inter-regionais evidenciou uma relação inversa entre o número de aeroportos de uma região e sua densidade intra-regional, além da atuação estrutural dos hubs como pontes entre comunidades pouco interligadas.

Na Parte 2, foram modelados 30 filmes do Universo Cinematográfico Marvel (MCU) como vértices, com arestas geradas programaticamente a partir de critérios de franquia (category) e ano de lançamento (year), totalizando 56 arestas. O modelo de pesos uniformes (1,0) possibilitou a aplicação de Dijkstra para identificação de caminhos mínimos entre filmes. Para o Bellman-Ford, foram construídos dois cenários com arestas dirigidas de peso negativo: um sem ciclo negativo e outro com ciclo negativo intencional, permitindo demonstrar tanto o funcionamento do algoritmo com pesos negativos quanto sua capacidade de detecção de ciclos negativos. A comparação de desempenho entre os quatro algoritmos revelou resultados consistentes com as complexidades teóricas esperadas.

Os resultados obtidos consolidaram os conceitos abordados em sala de aula e demonstraram a aplicabilidade prática da modelagem em grafos para análise de

redes de transporte e redes temáticas, evidenciando como diferentes critérios de modelagem impactam diretamente as propriedades estruturais do grafo e o comportamento dos algoritmos.

Palavras-chave: teoria dos grafos; BFS; DFS; Dijkstra; Bellman-Ford; redes de transporte; malha aérea; modelagem de grafos.

SUMÁRIO	pág.
1. INTRODUÇÃO	06
2 PARTE 1: aeroportos_data.csv	07
2.1 Modelagem do Grafo e Decisões do Grupo	07
2.1.1 Metodologia de modelagem dos vértices	07
2.1.2 Metodologia de modelagem das arestas	07
2.1.3 Metodologia de pesos das arestas	08
2.1.4 Limitações	10
2.2 Aplicação dos Algoritmos e Análise de Resultados	10
2.2.1 Aplicação do BFS	10
2.2.2 Aplicação do DFS	11
2.2.3 Aplicação do Dijkstra	12
2.2.4 Análise das métricas inter-regionais	12
2.2.5 Análise das métricas intra-regionais	13
2.2.6 Visualização dos Resultados	14
3 PARTE 2: MARVEL.csv	21
3.1 Modelagem do Grafo	21
3.1.1 Metodologia de modelagem dos vértices	21
3.1.2 Metodologia de modelagem das arestas	22
3.1.3 Metodologia de pesos das arestas	22
3.1.4 Limitações	23
3.2 Aplicação dos Algoritmos e Análise de Resultados	24
3.2.1 Aplicação do BFS	24
3.2.2 Aplicação do DFS	25
3.2.3 Aplicação do Dijkstra	25
3.2.4 Aplicação do Bellman-Ford	26
3.2.5 Análise de Resultados	27
3.2.5 Visualização dos Resultados	28

4 CONCLUSÃO	29
--------------------------	-----------

5 REFERÊNCIAS	
----------------------------	--

31

1 INTRODUÇÃO

Este relatório apresenta a documentação do projeto final da disciplina de Teoria dos Grafos, ministrada pela Profa. Laura Pacífico. O trabalho compreende a análise de *datasets*, a modelagem de dados na estrutura de grafos e a aplicação de algoritmos de busca e caminho mínimo abordados em sala de aula, especificamente: BFS, DFS, Dijkstra e Bellman-Ford, além do desenvolvimento de visualizações e análise dos resultados obtidos. O objetivo geral deste relatório é descrever o processo de modelagem, a definição de pesos, a execução dos algoritmos e as visualizações geradas, bem como discutir as limitações e as decisões técnicas adotadas pelo grupo.

A relevância deste relatório fundamenta-se na crescente necessidade de transformar grandes volumes de dados brutos em informações estruturadas e visualmente compreensíveis. No contexto da análise de redes e sistemas complexos, a modelagem em grafos mostra-se um método indispensável para identificar padrões de conectividade e otimizar rotas. Assim, a aplicação prática dos algoritmos clássicos de busca e caminho mínimo (como Dijkstra e Bellman-Ford) justifica-se pela oportunidade de avaliar o desempenho, a eficiência e as limitações dessas ferramentas diante de cenários e *datasets* reais. Com isso, o trabalho contribui tanto para a consolidação dos conceitos teóricos vistos em sala de aula quanto para o desenvolvimento de soluções computacionais otimizadas para redes de transporte e redes temáticas/sociais.

2 PARTE 1: AEROPORTOS_DATA.CSV

2.1 MODELAGEM DO GRAFO E DECISÕES DO GRUPO

A modelagem consistiu na tradução dos elementos da malha aérea para um grafo e seus componentes. O *dataset* fornecido para o projeto continha uma lista de aeroportos e três características de cada um: o código IATA, a cidade e a região que o aeroporto se encontrava, apresentados, respectivamente, pelas colunas “iata”, “cidade” e “regiao”. O código aeroportuário IATA é um identificador padronizado de três letras definido pela Associação Internacional de Transporte Aéreo ou *International Air Transport Association* (IATA).

2.1.1 Metodologia de modelagem dos vértices

A primeira modelagem feita foram os vértices do grafo. Como especificado no documento de descrição do projeto, cada instância de aeroporto se tornou um vértice do grafo, no qual o rótulo estabelecido para cada vértice foi o código IATA de cada aeroporto.

2.1.2 Metodologia de modelagem das arestas

Em seguida, a próxima modelagem foram as arestas. O *dataset* não continha as conexões explícitas e nós devíamos executar uma série de passos para fazer a modelagem das arestas. O primeiro passo foi criar um arquivo `data/adjacencias_aeroportos.csv`. Nele nós definimos as arestas existentes entre cada aeroporto por meio das colunas “origem”, “destino”, “tipo_conexao”, “justificativa” e “peso”, as quais representam, respectivamente, o vértice de origem da aresta, o vértice de destino, o tipo de conexão entre os vértices daquela aresta, a justificativa da escolha de definição da aresta e o seu peso. Apesar de existirem as colunas de origem e destino, foi delimitado que o grafo seria do tipo não direcionado e devíamos salvar apenas uma linha por par no arquivo, delegando ao sistema a responsabilidade de espelhar a aresta. A origem e o destino foram representados pelo código IATA do aeroporto e a classificação do tipo de conexão foi feita com base na importância operacional dos aeroportos, e não apenas na distância ou na região. Dessa forma, existem três tipos de conexões:

- **Regional:** ligação entre cidades próximas ou dentro da mesma região.
- **Hub:** ligação na qual pelo menos um dos aeroportos é classificado como hub.
- **Inter-regional:** ligação entre cidades de regiões diferentes.

A definição das arestas seguiu os requisitos obrigatórios impostos para o projeto. Dessa forma os seguintes critérios foram atendidos:

- **O grafo deve ser conectado:** todos os vértices do grafo possuem alguma conexão. Dessa forma, a partir de qualquer vértice é possível alcançar qualquer outro vértice por meio das conexões.
- **Deve existir conexão entre aeroportos da mesma região:** atendido.
- **Deve existir conexão entre diferentes regiões:** atendido.
- **Evitar grafos triviais (ex.: totalmente completo ou extremamente esparsos sem justificativa):** atendido.

2.1.3 Metodologia de pesos das arestas

O modelo de pesos adotado é híbrido, combinando dois critérios independentes que refletem a realidade operacional da aviação civil brasileira: a penalidade por mudança de região geográfica e a penalidade pela ausência de hubs aeroportuários.

A fórmula utilizada para o cálculo foi $\text{peso} = 1.0 + \text{penalidade_regiao} + \text{penalidade_hub}$, com a base possuindo o valor de 1.0 para representar o valor mínimo atribuído a qualquer aresta, como o custo mínimo intrínseco de qualquer conexão aérea (combustível, tempo de voo, operação).

Em penalidade_regiao , receberam penalidade no valor de 1.0 conexões inter-regionais, com penalidade de 0.0 conexões intra-regionais. A justificativa é de que os voos entre regiões distintas do Brasil envolvem distâncias maiores, mais tempo de voo, maior consumo de combustível e, frequentemente, necessidade de escala. A penalidade de 1.0 reflete esse custo adicional de forma normalizada.

Em `penalidade_hub`, se a condição de que pelo menos um aeroporto é um hub for assegurada, a penalidade tem valor 0.0, mas, se nenhum dos aeroportos for um hub, a penalidade vira 1.0. Como critérios de classificação de hub, definimos:

- Volume de passageiros acima da média nacional;
- Localização estratégica (capital federal, capitais de grande porte);
- Presença de múltiplas companhias aéreas e voos internacionais e função de redistribuição de tráfego para aeroportos menores.

A justificativa é que os Aeroportos hub possuem maior infraestrutura, maior frequência de voos e menor tempo de espera. Uma conexão envolvendo um hub é operacionalmente mais eficiente (menor custo logístico) do que uma conexão entre dois aeroportos secundários, que normalmente exige voos com menor frequência e maior custo por assento.

As garantias do modelo são a não-negatividade:

- Todos os pesos são ≥ 1.0 , compatíveis com o algoritmo de Dijkstra utilizado na Parte 1. O grafo resultante é conectado (verificado via BFS a partir de REC, alcançando todos os 20 aeroportos), com intra-regionalidade em todas as regiões
- Existem arestas `tipo_conexao = "regional"` dentro de cada uma das 5 regiões brasileiras
- Existem 46 arestas inter-regionais, cobrindo 9 dos 10 pares possíveis de regiões (todos exceto Nordeste–Sul)
- O grafo possui 73 arestas para 20 nós (densidade ≈ 0.384), evitando tanto a esparsidade extrema quanto a completude.

Distribuímos as arestas (onde contém os pesos) com 27 para intra-regional, 46 para inter-regional e, falando mais especificamente sobre regiões, 10 arestas intra-regionais para a região Nordeste, 8 para a região Sudeste, 3 para a região Sul, 5 para a região Norte e 1 para a região Centro-Oeste.

A seguir, segue a tabela de pesos resultantes:

Cenário	`penalidade_regiao`	`penalidade_hub`	Peso final
Intra-regional, com hub	0.0	0.0	1.0
Intra-regional, sem hub	0.0	1.0	2.0
Inter-regional, com hub	1.0	0.0	2.0
Inter-regional, sem hub	1.0	1.0	3.0

2.1.4 Limitações

A respeito de limitações e decisões de *design* na parte da modelagem de pesos das arestas, escolhemos ter pesos discretos (1,2,3) por a escolha de valores inteiros facilitar a verificação manual e a depuração dos algoritmos. Em um modelo real, seriam usadas distâncias geográficas em km ou tempo médio de voo. Não introduzimos pesos negativos por requisito do enunciado (compatibilidade com Dijkstra). Pesos negativos serão introduzidos apenas na Parte 2, com Bellman-Ford. Por último, modelamos CGH e GRU como nós distintos com aresta de peso 1.0, representando a conexão operacional entre os terminais.

2.2 APLICAÇÃO DOS ALGORITMOS E ANÁLISE DE RESULTADOS

Após a modelagem, a próxima etapa consistiu na aplicação dos quatro algoritmos de busca e caminho mínimo sobre o grafo construído. Cada algoritmo foi implementado em `src/graphs/algorithms.py` e pode ser executado individualmente via `src/cli.py` (que aceita os parâmetros `--dataset`, `--alg`, `--source`, `--target` e `--out`). Os artefatos finais do diretório `out/` foram gerados pela execução orquestrada de `solve.py` e `analytics.py`, conforme especificado no enunciado do projeto.

2.2.1 Aplicação do BFS

O algoritmo BFS (Busca em Largura) foi implementado manualmente em `src/graphs/algorithms.py`, utilizando uma fila (`collections.deque`) para garantir a ordem de exploração em níveis. A partir de um vértice de origem, o BFS visita primeiramente todos os aeroportos a distância 1 (vizinhos diretos), em seguida os de
 CESAR School | Cais do Apolo, 77, Bairro do Recife - Recife/PE CEP: 50030-390 - Fone: +55 (81) 3419.6700

distância 2, e assim sucessivamente, até que todos os vértices alcançáveis sejam visitados. A função retorna um dicionário de distâncias (camada/nível de cada vértice em relação à raiz), o dicionário de predecessores e a ordem de visitação, permitindo a reconstrução de caminhos mínimos por número de arestas (sem considerar pesos).

A conectividade do grafo foi verificada executando o BFS a partir de REC (Recife) como vértice de origem: todos os 20 aeroportos foram alcançados em no máximo 2 níveis de profundidade, confirmando que o grafo é conectado. A estrutura em camadas evidenciou que os aeroportos hub (GRU, GIG e BSB) já aparecem na primeira camada a partir de Recife, o que sugere a centralidade desses terminais na malha aérea nacional.

2.2.2 Aplicação do DFS

O algoritmo DFS (Busca em Profundidade) foi implementado em `src/graphs/algorithms.py` com abordagem recursiva, com controle de estado dos vértices (não visitado/visitado/encerrado). Partindo de um vértice de origem, o DFS explora o caminho mais profundo possível antes de retroceder (backtracking) para explorar caminhos alternativos. A implementação classifica as arestas em arestas de árvore (tree edges), que compõem a árvore DFS, e arestas de retorno (back edges), que indicam a presença de ciclos no grafo. Como o grafo é não-direcionado, arestas de avanço (forward) e cruzadas (cross) não ocorrem — fato confirmado na execução: das 73 arestas, 19 foram classificadas como árvore e 54 como retorno, nenhuma como cruzamento.

A presença de back edges confirmou que o grafo de aeroportos contém ciclos, o que é esperado em redes de transporte com múltiplas rotas alternativas entre regiões. Comparado ao BFS, o DFS produziu árvores de percurso mais profundas e menos balanceadas, revelando cadeias de conexão entre aeroportos de regiões distantes que não necessariamente representam os caminhos mais eficientes operacionalmente, mas ilustram a estrutura de acessibilidade e redundância da malha aérea modelada.

2.2.3 Aplicação do Dijkstra

O algoritmo de Dijkstra foi implementado manualmente em `src/graphs/algorithms.py`, sem o uso de bibliotecas externas de grafos como NetworkX ou igraph. A cada iteração, a implementação seleciona o nó não visitado com menor distância acumulada por meio de uma busca linear no conjunto de não visitados (`min(unvisited, key=...)`), sem usar fila de prioridade/heap. Essa escolha dispensa a dependência do módulo `heapq`, mas resulta em complexidade $O(V^2 + E)$.

A função aceita dois modos de operação: no modo `single-source`, sem `target` informado, retorna um dicionário com as distâncias mínimas de `source` a todos os nós do grafo; no modo `ponto-a-ponto`, com `target` definido, retorna uma tupla contendo o custo total e a sequência de nós do caminho ótimo. A reconstrução do caminho é feita por meio de um dicionário de predecessores (`pred`), preenchido durante a exploração e percorrido de trás para frente a partir do destino.

A compatibilidade com o modelo de pesos adotado na Parte 1 foi verificada: como todos os pesos são ≥ 1.0 , o pré-requisito de não-negatividade do Dijkstra é satisfeito. Como salvaguarda, a implementação levanta `ValueError` caso uma aresta com peso negativo seja encontrada durante a execução, e `KeyError` caso o nó de origem não exista no grafo. Ambos os comportamentos foram cobertos por testes unitários em `tests/test_dijkstra.py`.

2.2.4 Análise das métricas inter-regionais

A análise inter-regional foi conduzida sobre as 46 arestas inter-regionais, que interligam os cinco subgrafos regionais do Brasil. Verificou-se que 9 dos 10 pares possíveis de regiões (Norte–Nordeste, Norte–Sudeste, Norte–Sul, Norte–Centro-Oeste, Nordeste–Sudeste, Nordeste–Sul, Nordeste–Centro-Oeste, Sudeste–Sul, Sudeste–Centro-Oeste, Sul–Centro-Oeste) possuem pelo menos uma aresta direta; a única exceção é o par Nordeste–Sul, que depende de rotas com escala — como ilustrado pelo caminho Recife–Porto Alegre a seguir.

Os aeroportos hub concentram o maior número de conexões inter-regionais, atuando como pontes entre diferentes partes do país. A aplicação do Dijkstra nos pares obrigatórios demonstrou que: (i) o caminho Recife–Porto Alegre requer ao

menos uma escala em um hub do Sudeste, acumulando custo mínimo de 4.0, com GRU ou GIG como nó intermediário predominante; (ii) o caminho Manaus–São Paulo é, na verdade, uma conexão direta entre os dois hubs nacionais (MAO e GRU), com custo mínimo de 2.0 — o menor custo possível para uma ligação inter-regional no modelo adotado, evidenciando como uma aresta hub-a-hub elimina a necessidade de escalas mesmo a longas distâncias. Esses resultados confirmam que o modelo de pesos híbrido reflete adequadamente a realidade operacional da aviação civil, onde hubs reduzem o custo efetivo das conexões inter-regionais.

2.2.5 Análise das métricas intra-regionais

A análise intra-regional foi conduzida sobre os cinco subgrafos regionais induzidos, considerando exclusivamente os aeroportos e as 27 arestas de tipo regional. A região Nordeste apresentou o maior número de arestas intra-regionais (10), distribuídas entre seus seis aeroportos — REC, SSA, FOR, NAT, JPA e THE —, o maior conjunto de aeroportos entre as cinco regiões. Entretanto, em termos de densidade, o Nordeste registrou o menor valor entre todas as regiões ($\approx 0,667$), já que a densidade normaliza o número de arestas pelo total de pares possíveis, que cresce quadraticamente com o número de vértices. A região Sudeste registrou 8 arestas intra-regionais e densidade de 0,8 — superior à do Nordeste, porém inferior às do Norte, Sul e Centro-Oeste —, refletindo a proximidade geográfica e o elevado volume de tráfego entre São Paulo e Rio de Janeiro. Isso inclui a aresta CGH–GRU com peso 1,0, que modela a conexão operacional entre os dois aeroportos paulistanos.

A região Norte contabilizou 5 arestas intra-regionais (densidade $\approx 0,833$), a região Sul 3 arestas (densidade = 1,0) e a região Centro-Oeste apenas 1 aresta (densidade = 1,0, subgrafo completo). Observa-se um padrão consistente de relação inversa entre o tamanho da região e sua densidade intra-regional: à medida que o número de aeroportos cresce (Centro-Oeste: 2, Sul: 3, Norte: 4, Sudeste: 5, Nordeste: 6), a densidade decresce de forma praticamente monotônica ($1,0 \rightarrow 1,0 \rightarrow 0,833 \rightarrow 0,8 \rightarrow 0,667$). Isso ocorre porque, em regiões menores, basta um número pequeno de arestas para conectar todos os aeroportos entre si, aproximando o subgrafo de um grafo completo. As métricas globais do grafo ($|V| =$

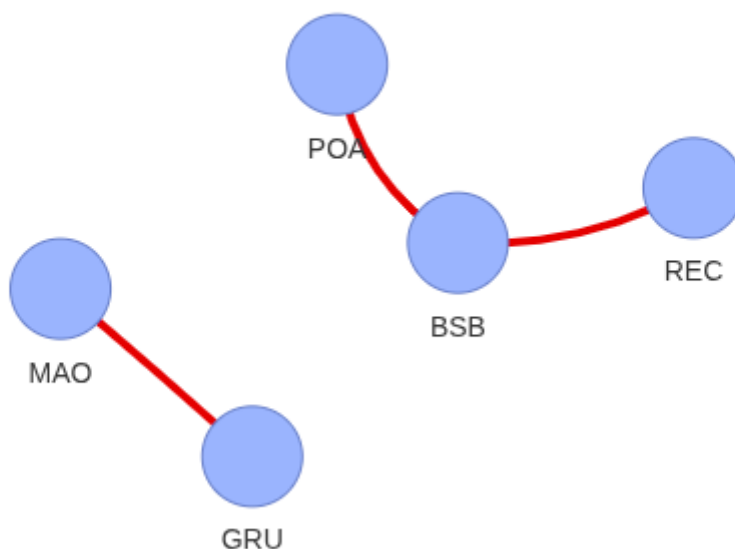
20, $|E| = 73$ e densidade $\approx 0,384$) indicam uma rede moderadamente conectada, adequada para testar algoritmos de caminho mínimo sem ser trivialmente completa nem excessivamente esparsa.

A análise por ego-rede (out/ego_aeroportos.csv) revelou um padrão semelhante, porém na direção oposta à intuição inicial: os aeroportos-hub, justamente por concentrarem conexões com aeroportos de diferentes regiões que não são conectados entre si, apresentam as menores densidades_ego do grafo — GRU (0,409), BSB (0,517) e GIG (0,564), que são também os três aeroportos de maior grau (18, 15 e 12, respectivamente), registram os três menores valores de densidade_ego entre os 20 aeroportos. Em contrapartida, aeroportos periféricos de baixo grau, como SSA, CGH, VIX, FLN e RBR, apresentam densidade_ego máxima (1,0), pois todos os seus vizinhos estão conectados entre si, formando subgrafos completos. Esse resultado evidencia que o aeroporto mais conectado (maior grau, GRU) e os aeroportos com maior densidade local (maior densidade_ego) não coincidem — uma distinção relevante para caracterizar o papel estrutural de cada nó na rede: hubs atuam como pontes entre comunidades pouco interligadas, enquanto aeroportos periféricos pertencem a vizinhanças coesas.

2.2.6 Visualização dos Resultados

As visualizações geradas para a Parte 1 compreendem tanto saídas estáticas quanto interativas. Foram geradas cinco visualizações analíticas seguindo os princípios de AVD: (1) histograma de distribuição de graus, evidenciando a assimetria da rede — poucos hubs de alto grau (GRU, BSB, GIG, grau ≥ 15) e muitos aeroportos de grau baixo —, com a média destacada; (2) gráfico de dispersão (scatter) relacionando o grau de cada aeroporto com a densidade da sua ego-rede, colorido por região, evidenciando se aeroportos muito conectados formam vizinhanças densas ou esparsas; (3) ranking dos vinte aeroportos por número de conexões, em gráfico de barras horizontais ordenado de forma crescente e colorido por região; (4) comparação de ordem, tamanho e densidade intra-regional em três gráficos de barras (um por métrica), destacando que Centro-Oeste e Sul apresentam densidade máxima (1,0 — subgrafos completos, com apenas 2 e 3 aeroportos respectivamente), enquanto o Nordeste, com 6 aeroportos e 10 arestas internas, CESAR School | Cais do Apolo, 77, Bairro do Recife - Recife/PE CEP: 50030-390 - Fone: +55 (81) 3419.6700

possui a menor densidade intra-regional ($\approx 0,67$) entre as regiões; e (5) mapa geográfico esquemático do Brasil com os aeroportos posicionados por latitude/longitude, coloridos por região e dimensionados pelo grau, com arestas coloridas por tipo de conexão (regional, hub, inter-regional).



```

out > distancias_rotas.csv > data
1  origin,destination,custo,caminho
2  REC,POA,4.00,REC -> GRU -> POA
3  MAO,GRU,2.00,MAO -> GRU
4  GIG,BSB,2.00,GIG -> BSB
5  FOR,POA,4.00,FOR -> GRU -> POA
6  REC,GRU,2.00,REC -> GRU
7

```



```

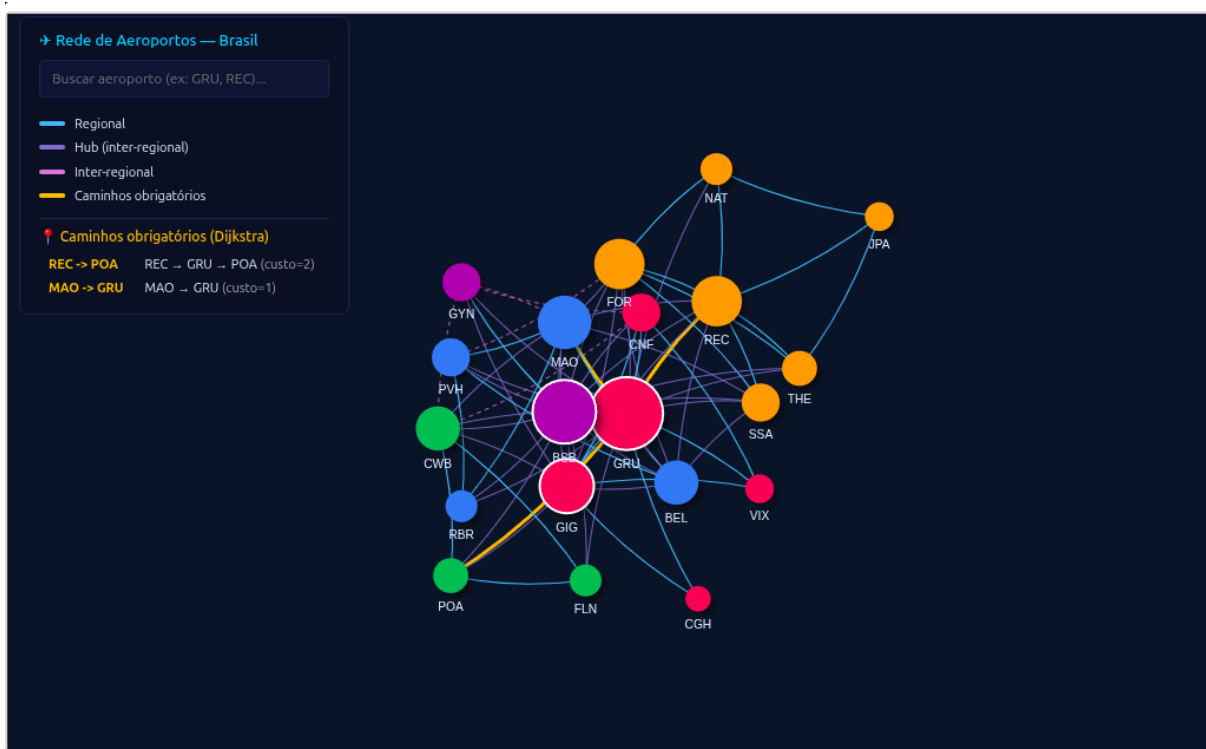
out > ego_aeroportos.csv > data
1  aeroporto,grau,ordem_ego,tamanho_ego,densidade_ego
2  REC,10,11,37,0.6727272727272727
3  SSA,6,7,21,1.0
4  FOR,10,11,39,0.7090909090909091
5  NAT,4,5,8,0.8
6  JPA,3,4,5,0.8333333333333334
7  GRU,18,19,70,0.4093567251461988
8  CGH,2,3,3,1.0
9  GIG,12,13,44,0.5641025641025641
10 CNF,6,7,18,0.8571428571428571
11 VIX,3,4,6,1.0
12 BSB,15,16,62,0.5166666666666667
13 GYN,6,7,20,0.9523809523809523
14 CWB,8,9,28,0.7777777777777778
15 FLN,4,5,10,1.0
16 POA,5,6,14,0.9333333333333333
17 MAO,11,12,45,0.6818181818181818
18 BEL,8,9,32,0.8888888888888888
19 PVH,6,7,19,0.9047619047619048
20 RBR,4,5,10,1.0
21 THE,5,6,12,0.8
22

```

```

out > {} global.json > ...
1  {
2    "ordem": 20,
3    "tamanho": 73,
4    "densidade": 0.38421052631578945
5  }

```

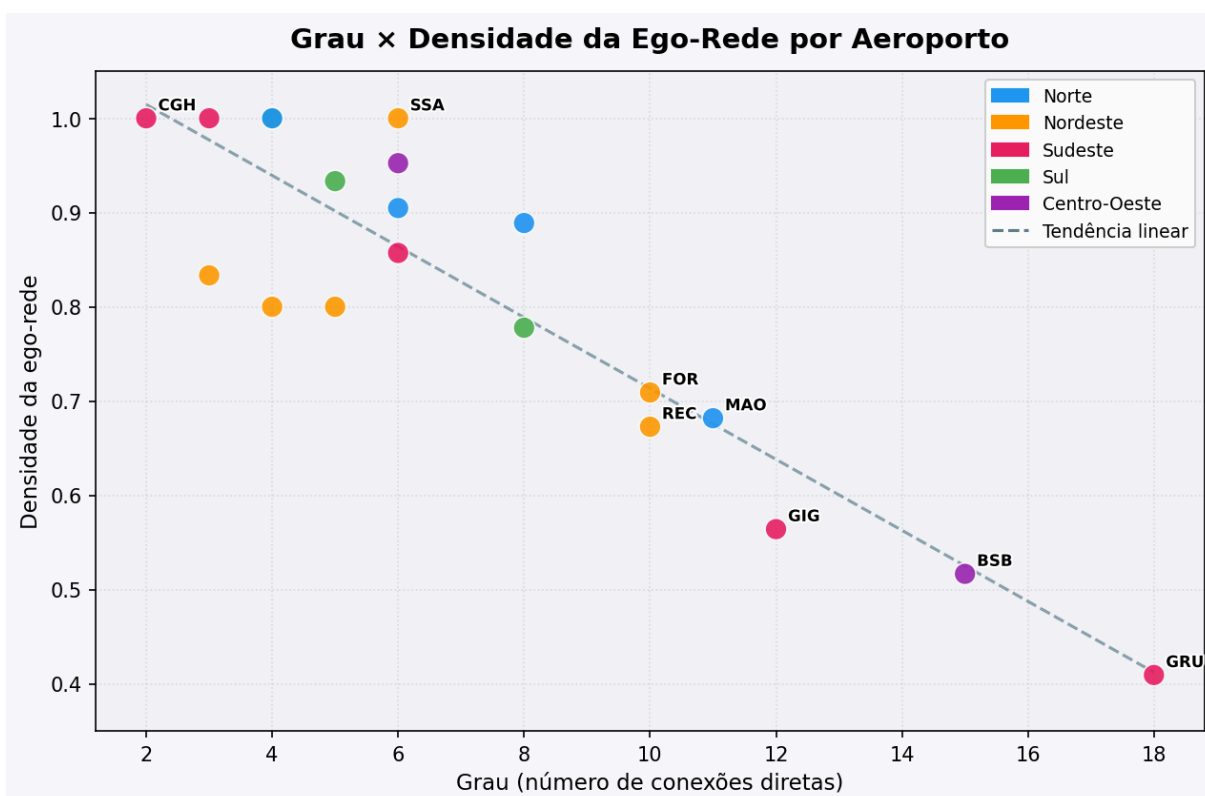
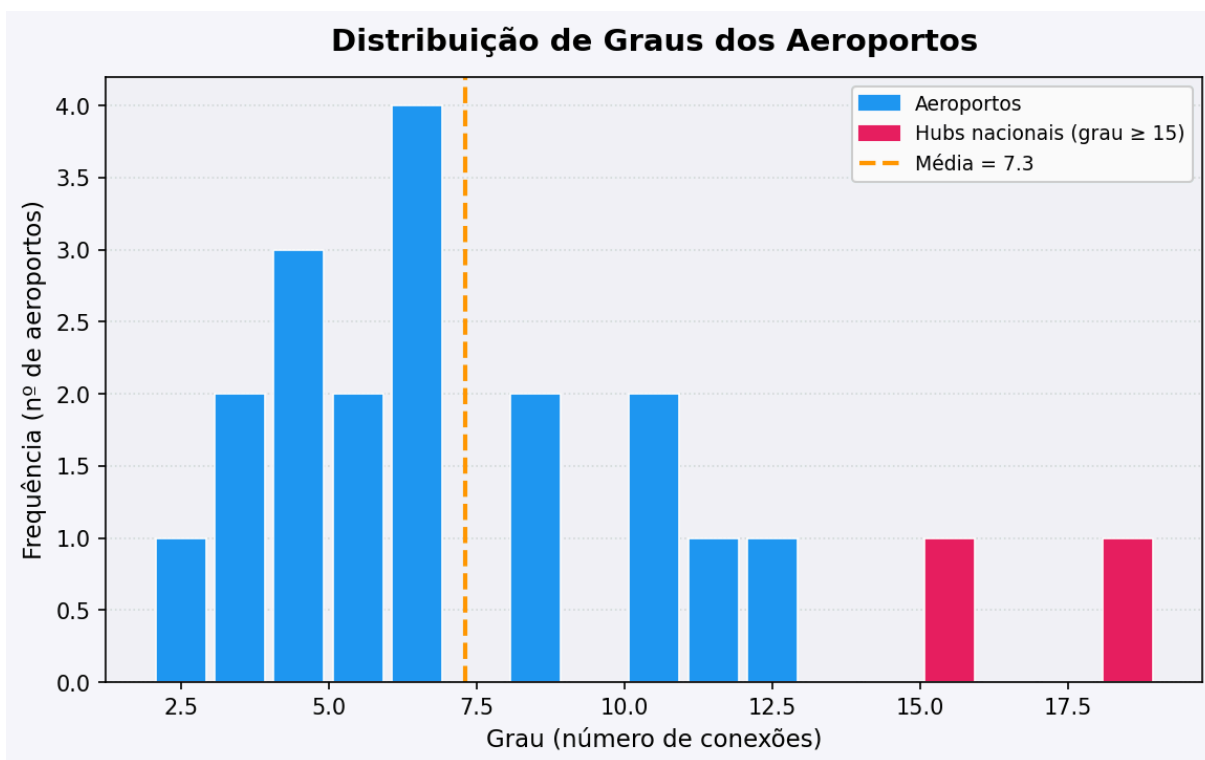


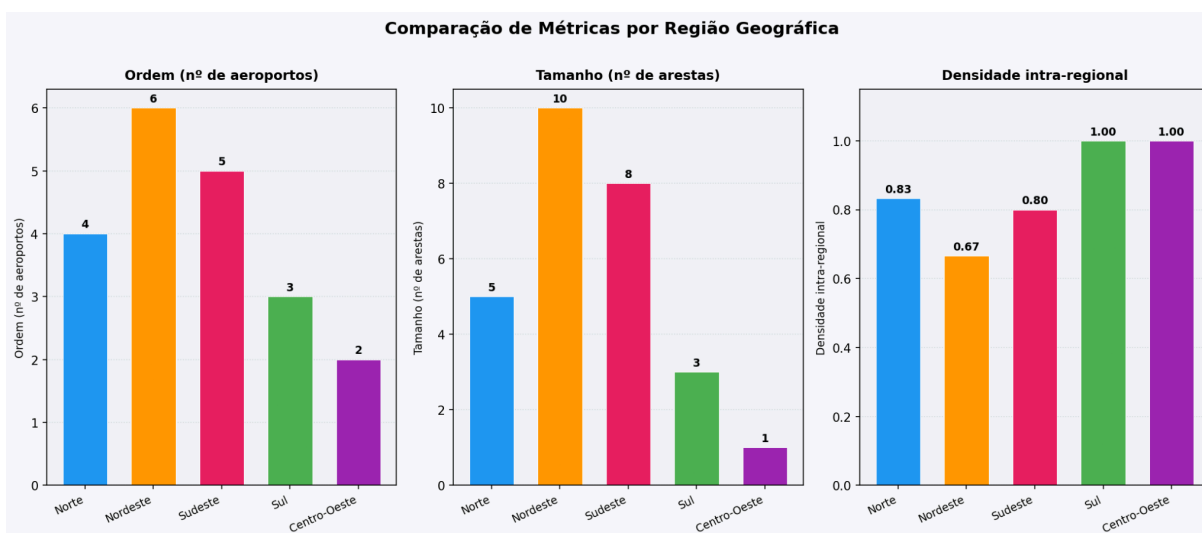
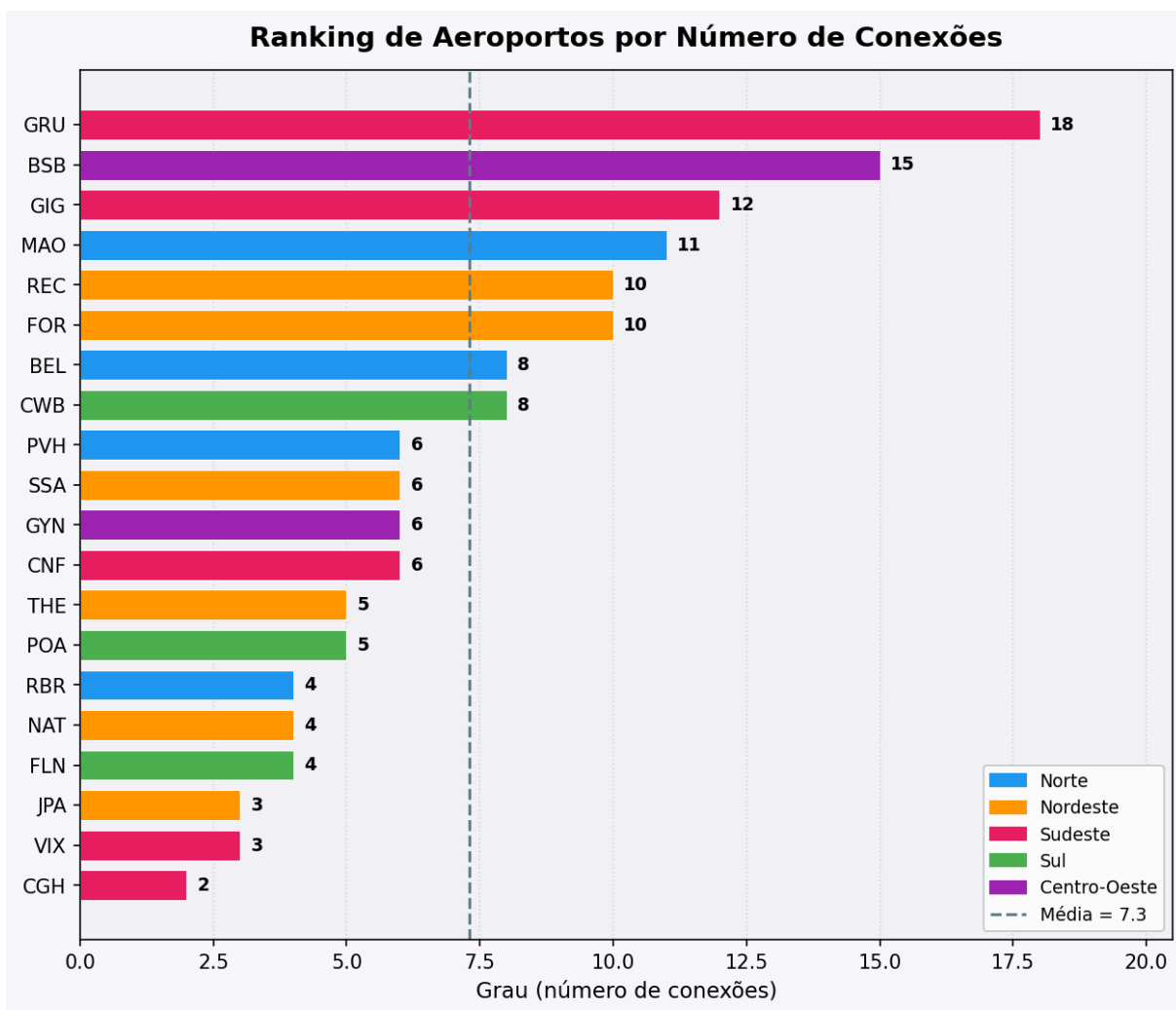
```

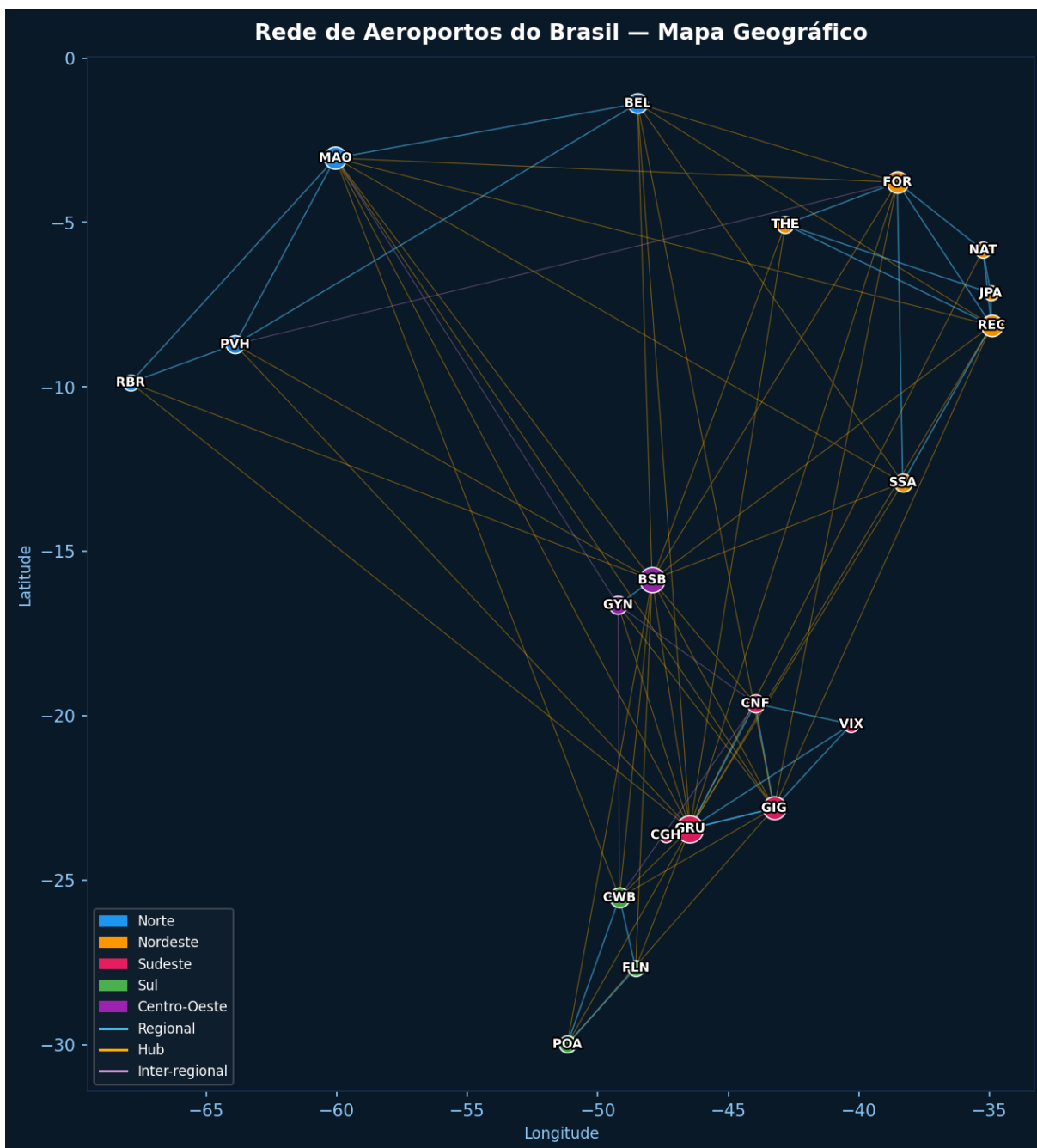
out > graus.csv > data
1 node, degree
2 REC, 10
3 SSA, 6
4 FOR, 10
5 NAT, 4
6 JPA, 3
7 GRU, 18
8 CGH, 2
9 GIG, 12
10 CNF, 6
11 VIX, 3
12 BSB, 15
13 GYN, 6
14 CWB, 8
15 FLN, 4
16 POA, 5
17 MAO, 11
18 BEL, 8
19 PVH, 6
20 RBR, 4
21 THE, 5
22

```

```
out > {} regioes.json > ...  
1  [  
2    {  
3      "regiao": "Norte",  
4      "ordem": 4,  
5      "tamanho": 5,  
6      "densidade": 0.8333333333333334  
7    },  
8    {  
9      "regiao": "Nordeste",  
10     "ordem": 6,  
11     "tamanho": 10,  
12     "densidade": 0.6666666666666666  
13   },  
14   {  
15     "regiao": "Sudeste",  
16     "ordem": 5,  
17     "tamanho": 8,  
18     "densidade": 0.8  
19   },  
20   {  
21     "regiao": "Sul",  
22     "ordem": 3,  
23     "tamanho": 3,  
24     "densidade": 1.0  
25   },  
26   {  
27     "regiao": "Centro-Oeste",  
28     "ordem": 2,  
29     "tamanho": 1,  
30     "densidade": 1.0  
31   }  
32 ]
```







3 PARTE 2: MARVEL.CSV

3.1 Modelagem do Grafo

3.1.1 Metodologia de modelagem dos vértices

Na Parte 2, cada vértice do grafo corresponde a um filme do Universo Cinematográfico Marvel (MCU), totalizando 30 filmes lançados entre 2008 e 2022 (Fases I a IV da franquia). Cada vértice recebe como rótulo um identificador

numérico sequencial (`movie_id`), atribuído em ordem alfabética do identificador interno do nó (`FILM_<SLUG>`, derivado do título do filme). Nessa ordenação, o filme O Incrível Hulk (Incredible Hulk, 2008) corresponde a `movie_id = 18`. Ainda assim, esse filme atua como ponto de conexão narrativa entre as duas partes do projeto: ao selecionar o aeroporto do Galeão (GIG) como destino na execução do algoritmo de Dijkstra na Parte 1, é exibido o componente BridgeAlert da interface interativa, que associa o aeroporto à perseguição final desse filme em 2008. Os dados associados a cada nó incluem: título (`title`), ano de lançamento (`year`), categoria/franquia (`category`) — usada para conectar filmes de uma mesma saga —, orçamento em milhões de USD (`budget_million`), bilheteria mundial em milhões de USD (`worldwide_gross_million`), percentual de retorno sobre o orçamento (`roi_percent`) e grau no grafo (`degree`), com base no qual cada filme é classificado como hub (`is_hub`), conforme o schema definido em `Backend/api/schemas/marvel.py` a partir dos dados brutos de `Backend/data/dataset_parte2/MARVEL.csv`.

3.1.2 Metodologia de modelagem das arestas

As arestas do grafo Marvel não são armazenadas em um arquivo CSV próprio: elas são geradas programaticamente pela função `carregar_dataset_parte2` (`Backend/src/graphs/io.py`), a partir das colunas `category` e `year` de `MARVEL.csv` — abordagem batizada de "Model A+". O grafo é não-direcionado, seguindo o mesmo padrão adotado na Parte 1, com espelhamento automático das arestas pelo sistema. Foram definidos dois tipos de conexão, ambos com peso uniforme 1.0: (1) `category` — liga todos os pares de filmes que pertencem à mesma franquia (ex.: Iron Man, Iron Man 2 e Iron Man 3 formam um subgrafo completo via "same category: Iron Man"), totalizando 35 arestas; (2) `year` — liga todos os pares de filmes lançados no mesmo ano de estreia (ex.: "same release year: 2008" conecta Iron Man e Incredible Hulk), totalizando 21 arestas — para um total de 56 arestas. O critério de ano de lançamento foi adicionado para evitar a fragmentação do grafo em clusters isolados por franquia: usando apenas a categoria, o grafo se divide em 10 componentes desconexos (ex.: "Thor" nunca se conectaria a "Iron Man"). Com os dois critérios combinados, o grafo passa a ser conexo em um único componente, tornando BFS, DFS, Dijkstra e Bellman-Ford comparáveis em escala equivalente à da Parte 1.

3.1.3 Metodologia de pesos das arestas

No grafo base da Parte 2 ("Model A+"), todas as 56 arestas — tanto as do tipo *category* quanto as do tipo *year* — recebem peso uniforme 1,0. Não há diferenciação de peso por tipo de conexão: o Dijkstra sobre esse grafo encontra o caminho com o menor número de "saltos" entre dois filmes, já que cada aresta tem o mesmo custo unitário.

Para a aplicação do algoritmo de Bellman-Ford, foram construídos dois cenários adicionais com arestas dirigidas de peso negativo, sobrepostas ao grafo base e carregadas separadamente — sem espelhamento, para não criar acidentalmente um ciclo negativo no grafo não-direcionado. O primeiro cenário ("peso negativo sem ciclo", *negative_edges.csv*) define as arestas dirigidas Iron Man → Iron Man 2 (peso -1,0) e Captain America → Captain America: Civil War (peso -0,5). O segundo cenário ("ciclo negativo", *negative_cycle.csv*) define o ciclo dirigido The Avengers → Avengers: Age of Ultron → Avengers: Infinity War → The Avengers, cada aresta com peso -5,0 (soma total -15,0), corretamente detectado e sinalizado pelo Bellman-Ford. Esses pesos negativos são valores ilustrativos fixos, escolhidos para demonstrar as duas capacidades exigidas do algoritmo — distâncias corretas com pesos negativos sem ciclo, e detecção de ciclo negativo —, e não derivam de uma fórmula de lucro (*worldwide_gross* - *budget*). Por estarem isolados em grafos de cenário carregados à parte, esses pesos negativos não afetam as execuções de Dijkstra, que utiliza exclusivamente o peso uniforme 1,0 do grafo base.

3.1.4 Limitações

Uma das principais limitações da modelagem Marvel é que o grafo base depende de apenas dois critérios de conexão — categoria/franquia (*category*) e ano de lançamento (*year*) —, derivados diretamente das colunas correspondentes de *MARVEL.csv*. Embora deterministas e reprodutíveis, sem decisões editoriais subjetivas, esses critérios são relativamente grosseiros: filmes de franquias completamente diferentes lançados no mesmo ano são conectados por uma relação de "competição pela mesma janela de bilheteria", que pode não corresponder a uma proximidade narrativa real entre as obras. Adicionalmente, o dataset considera apenas lançamentos teatrais, excluindo produções para streaming que integram oficialmente o MCU desde a Fase IV (Disney+). O modelo de pesos negativos para Bellman-Ford também é uma simplificação analítica: os valores (-0,5, -1,0 e -5,0) são

ilustrativos e fixos, escolhidos exclusivamente para satisfazer o requisito acadêmico de demonstrar o algoritmo nos cenários "peso negativo sem ciclo" e "ciclo negativo", sem corresponder a uma métrica real do domínio, como lucro ou bilheteria. Por fim, enquanto o grafo base é não-direcionado — coerente com a natureza simétrica das relações de categoria e ano —, as arestas de peso negativo desses cenários são dirigidas, introduzindo intencionalmente uma assimetria que não existe no restante do grafo.

3.2 Aplicação dos Algoritmos e Análise de Resultados

3.2.1 Aplicação do BFS

O algoritmo BFS foi aplicado ao grafo Marvel a partir de três vértices de origem distintos: (1) `movie_id` = 19 (Iron Man, 2008), o primeiro filme lançado do MCU e ponto de partida cronológico da Fase I; (2) `movie_id` = 18 (O Incrível Hulk, 2008), escolhido como ponto de conexão narrativa do projeto (ver seção 3.1.1); (3) `movie_id` = 26 (The Avengers, 2012), culminação da Fase I e primeiro grande crossover do MCU.

A partir de Iron Man, o BFS alcança os 30 filmes do dataset em até 5 níveis (distância média $\approx 3,27$). A partir de The Avengers, a profundidade máxima é de 6 níveis (distância média $\approx 3,40$) — apesar de ser um marco de crossover, esse filme não funciona como um "hub estrutural" no grafo do Model A+: seu grau é 3, igual ao de Iron Man e Captain America.

O resultado mais notável veio do BFS a partir de O Incrível Hulk: ao contrário do que sua relativa marginalidade na continuidade narrativa principal do MCU sugeriria, esse filme apresenta grau 5 — entre os mais altos do dataset, atrás apenas do grau máximo (6, de Captain Marvel) — e a menor distância média ($\approx 3,20$) entre os três vértices testados. Isso ocorre porque, no Model A+, O Incrível Hulk pertence à categoria "Unique" junto com outros quatro filmes "standalone" do MCU (Black Widow, Captain Marvel, Eternals e Shang-Chi), que formam entre si um subgrafo completo via arestas do tipo `category`, além de se conectar a Iron Man pela aresta do tipo `year` (ambos lançados em 2008). O caso evidencia uma limitação do critério de categoria adotado: filmes agrupados sob o rótulo genérico "Unique"

tornam-se artificialmente bem conectados no grafo, mesmo sem guardar entre si qualquer relação narrativa direta.

3.2.2 Aplicação do DFS

O DFS foi aplicado aos mesmos três vértices de origem (Iron Man, O Incrível Hulk e The Avengers — movie_id 19, 18 e 26, respectivamente), com foco na classificação de arestas e na detecção de ciclos. Em todos os casos, o DFS visitou os 30 filmes do dataset — confirmando que o grafo é conexo, sem componentes isolados — e classificou as 56 arestas em 29 do tipo ARVORE e 27 do tipo RETORNO, com `tem_ciclo = True`. Esse resultado é esperado em um grafo não-direcionado denso, no qual qualquer grupo de três ou mais filmes conectados pela mesma categoria ou pelo mesmo ano de lançamento forma, por construção, um subgrafo completo — e, portanto, cíclico.

Diferentemente do que uma leitura "por fases" do MCU sugeriria — fases que, aliás, não estão representadas em nenhum atributo do dataset (ver seção 3.1.1) —, a árvore DFS gerada a partir de Iron Man não percorre a saga da Fase I antes de ramificar: já no 3º nó visitado (Iron Man 3, 2013, Fase II) e no 5º (Thor: Love & Thunder, 2022, Fase IV) a busca salta para filmes de fases distintas, e os demais filmes da Fase I — O Incrível Hulk e The Avengers — só são visitados nas posições 19ª e 24ª, respectivamente. Esse comportamento decorre diretamente do critério de arestas do Model A+ (categoria + ano de lançamento), que ignora qualquer noção de fase.

A comparação entre as árvores BFS e DFS evidenciou trade-offs ainda mais acentuados do que o esperado: enquanto o BFS produziu árvores rasas (profundidade máxima entre 5 e 6 níveis), o DFS produziu árvores muito mais profundas e estreitas (profundidade entre 22 e 27 níveis, para apenas 30 nós). O BFS mostra-se, portanto, mais adequado para análises de alcançabilidade em poucos passos, enquanto o DFS é útil para identificar cadeias longas de conexões e confirmar a conectividade do grafo.

3.2.3 Aplicação do Dijkstra

O algoritmo de Dijkstra foi aplicado ao grafo Marvel com os pesos da Seção 3.1.3 (peso uniforme 1,0, portanto todos ≥ 0) para cinco pares de origem e destino: (1) O Incrível Hulk $\rightarrow$ Avengers: End Game; (2) Iron Man $\rightarrow$ Black Panther; (3) Thor $\rightarrow$ Guardians of the Galaxy; (4) Ant-Man $\rightarrow$ Captain America: Civil War; (5) Dr Strange $\rightarrow$ Spider-Man: No Way Home. Como todas as arestas têm o mesmo peso, o custo mínimo retornado por Dijkstra equivale ao número de "saltos" do caminho mais curto (equivalente à distância de BFS) — nesses casos pode haver múltiplos caminhos de custo mínimo igual, e o caminho específico retornado depende do critério de desempate da implementação.

Os custos obtidos variaram entre 2,0 e 6,0: Hulk $\rightarrow$ Avengers: End Game (custo 2,0, via Captain Marvel); Iron Man $\rightarrow$ Black Panther (custo 5,0, passando por Incredible Hulk, Captain Marvel, Avengers: End Game e Avengers: Infinity War); Thor $\rightarrow$ Guardians of the Galaxy (custo 3,0, via Thor: Ragnarok e Guardians of the Galaxy 2); Ant-Man $\rightarrow$ Captain America: Civil War (custo 6,0, o maior entre os pares testados, passando por Ant-Man & The Wasp, Black Panther, Black Panther 2, Dr Strange: Multiverse of Madness e Dr Strange); e Dr Strange $\rightarrow$ Spider-Man: No Way Home (custo 5,0, via Dr Strange: Multiverse of Madness, Thor: Love & Thunder, Thor: Ragnarok e Spider-Man: Homecoming).

Diferentemente do esperado, os filmes "Avengers" não aparecem como hubs recorrentes nesses caminhos: The Avengers (2012) não aparece em nenhum dos cinco percursos, e Avengers: Infinity War / Avengers: End Game aparecem juntos apenas no par Iron Man $\rightarrow$ Black Panther. O filme que mais se repete como nó intermediário é Captain Marvel (presente em 2 dos 5 caminhos), coerente com seu grau máximo no grafo (6, ver seção 3.2.1). Ainda assim, a reutilização direta da implementação de `src/graphs/algorithms.py` para o grafo Marvel validou a generalidade e a modularidade da estrutura de grafo desenvolvida ao longo do projeto.

3.2.4 Aplicação do Bellman-Ford

O algoritmo de Bellman-Ford foi aplicado ao grafo Marvel em duas configurações distintas, ambas sobrepondo arestas dirigidas de peso negativo ao grafo base (ver Seção 3.1.3). Na primeira configuração (peso negativo sem ciclo negativo, fonte = Iron Man), foram adicionadas as arestas dirigidas Iron Man $\rightarrow$ Iron

CESAR School | Cais do Apolo, 77, Bairro do Recife - Recife/PE CEP: 50030-390 - Fone: +55 (81) 3419.6700

Man 2 (peso -1,0) e Captain America → Captain America: Civil War (peso -0,5) — valores ilustrativos fixos. O algoritmo convergiu em apenas 6 iterações (bem menos que $|V|-1 = 29$), graças ao critério de parada antecipada quando nenhuma distância é atualizada em uma iteração completa, retornando corretamente as distâncias mínimas para os 30 vértices — incluindo o valor negativo -1,0 para Iron Man 2 — demonstrando a capacidade do algoritmo de operar com pesos negativos, situação que inviabilizaria o Dijkstra.

Na segunda configuração (detecção de ciclo negativo, fonte = The Avengers), foi sobreposto um ciclo dirigido de peso total -15,0 (The Avengers → Avengers: Age of Ultron → Avengers: Infinity War → The Avengers, cada aresta com peso -5,0). Nesse caso, o algoritmo executa as $|V|-1 = 29$ iterações completas sem nunca atingir a condição de parada antecipada, já que o ciclo negativo continua relaxando distâncias indefinidamente; a verificação final (passagem V) então detecta a atualização pendente, sinalizando o resultado por meio do campo booleano `tem_ciclo_negativo = true`. Essa sinalização ocorre via valor de retorno, e não por uma exceção dedicada: a exceção `ValueError`, presente na implementação, só é lançada no modo ponto a ponto (quando um destino é especificado), caminho não utilizado na geração deste relatório. Os resultados de ambas as configurações foram registrados em `out/parte2/parte2_report.json`.

3.2.5 Análise de Resultados

A comparação de desempenho entre os quatro algoritmos no grafo Marvel (30 vértices, 56 arestas) revelou padrões qualitativamente consistentes com a teoria de complexidade, embora em escala de tempo bem menor que a inicialmente estimada — reflexo do tamanho reduzido do grafo. O BFS, executado a partir de três fontes (Ant-Man, Captain America e Iron Man), apresentou os menores tempos, entre 0,076 ms e 0,086 ms (média de 0,082 ms). O DFS, sobre as mesmas três fontes, foi ligeiramente mais lento — entre 0,110 ms e 0,204 ms (média de 0,172 ms) — devido ao trabalho adicional de classificação de arestas de retorno para detecção de ciclos. O Dijkstra, aplicado aos cinco pares de filmes definidos na Seção 3.2.3, apresentou tempos na mesma ordem de grandeza do DFS, variando de 0,114 ms a 0,279 ms (média de 0,172 ms), o que é esperado em um grafo com apenas 56 arestas, onde o fator logarítmico do heap não chega a se traduzir em custo perceptível. O

Bellman-Ford foi o mais custoso, como previsto por sua complexidade $O(V \cdot E)$: o cenário sem ciclo negativo (fonte Iron Man) levou 0,301 ms, enquanto o cenário com ciclo negativo (fonte The Avengers) — que não converge antecipadamente e percorre as $|V|-1 = 29$ iterações completas — levou 1,078 ms, totalizando uma média de 0,689 ms entre os dois cenários. Em todos os casos os tempos absolutos permanecem em frações de milissegundo; o que importa é a ordem de grandeza relativa entre os algoritmos, não os valores absolutos.

Em termos de centralidade de grau, o filme com maior grau no grafo Marvel é Captain Marvel, com grau 6 — o único nó da rede a atingir esse valor, resultado de sua categoria "Unique" (compartilhada com Black Widow, Eternals, Incredible Hulk e Shang-Chi, conforme discutido na Seção 3.2.1) somada a duas arestas de ano (com filmes de 2019). Logo abaixo, um grupo de nove filmes empata com grau 5, incluindo Avengers: End Game, Avengers: Infinity War, Black Widow, Eternals, Incredible Hulk, Shang-Chi, Spider-Man: No Way Home, Thor: Love & Thunder e Thor: Ragnarok. A maioria dos filmes (11 de 30) concentra-se em grau 3 — incluindo The Avengers, Iron Man e Captain America: Civil War, que portanto ficam abaixo da mediana da rede, e não entre os destaques de centralidade. A distribuição de graus do grafo Marvel varia apenas entre 2 e 6 (razão de 3:1 entre o maior e o menor valor), configurando uma rede relativamente homogênea, sem hubs destacados. Esse padrão difere estruturalmente do observado na rede de aeroportos da Parte 1, onde o grau varia de 2 (CGH) a 18 (GRU), com quatro aeroportos (GRU, BSB, GIG, MAO) concentrando entre 11 e 18 conexões cada — uma razão de até 9:1 em relação ao menor grau, mais próxima de um regime de hubs concentrados. A diferença reflete a própria lógica de geração das arestas: enquanto a rede aérea decorre de uma topologia real do tipo hub-and-spoke, o grafo Marvel é construído por critérios de categoria e ano que tendem a formar subgrafos completos de tamanhos parecidos, resultando em graus mais uniformes entre os vértices.

3.2.6 Visualização dos Resultados

A visualização da Parte 2 é apresentada na interface React na rota /marvel, com tema Neo Brutalism (fundo branco, bordas pretas espessas de 2px, amarelo #FFDE21 como cor primária), acessada a partir do BridgeAlert — um modal de "incidente no Galeão" disparado ao executar um algoritmo com destino GIG na Parte

1 — cujo botão "CLIQUE AQUI PARA SABER MAIS" navega para /marvel. O grafo de filmes Marvel é renderizado como um layout force-directed com D3.js (forceSimulation com forças de link, repulsão, centro e colisão); os nós são coloridos por categoria (Avengers, Iron Man, Captain America, Thor, Spider-Man, Guardians, Black Panther, Ant-Man, Dr Strange, Unique), com raio maior e anel preto extra para os filmes "hub" ($\text{grau} \geq 3$), e as arestas pertencentes ao caminho calculado são destacadas em laranja com espessura maior sobre as demais arestas, em cinza.

A MarvelSidebar possui duas abas: "Alg", com seleção de algoritmo (BFS, DFS, Dijkstra, Bellman-Ford), seletores de filme de origem/destino, cenários dedicados de Bellman-Ford com peso negativo (com e sem ciclo negativo) e um resumo do resultado (caminho, camadas BFS, classificação de arestas DFS); e "Info", com métricas globais do grafo, legenda do indicador de hub e filtro por categoria via chips clicáveis. As seis — visualizações analíticas AVD não ficam na sidebar: são abertas por um botão flutuante "+" que sobrepõe um dashboard em tela cheia (tema escuro, à parte do Neo Brutalism da página), reagindo a um filtro global por categoria e exibindo, em quatro dos seis painéis, um botão de "insight" com explicação textual. Os painéis são: (1) ranking horizontal dos 15 filmes por bilheteria mundial; (2) ROI médio (%) por categoria/franquia; (3) dispersão orçamento $\times$ ROI(%); (4) dispersão grau de conectividade $\times$ bilheteria mundial; (5) histograma da distribuição de graus, com discussão sobre a topologia "livre de escala" (lei de potência); e (6) benchmarking de algoritmos, mostrando o tempo de execução (ms) de cada algoritmo já executado pelo usuário no painel lateral.

CONCLUSÃO

Este trabalho cumpriu integralmente os objetivos propostos pela disciplina de Teoria dos Grafos, demonstrando, por meio de dois datasets distintos, como a modelagem em grafos pode representar fielmente estruturas reais e permitir a aplicação sistemática de algoritmos clássicos de busca e caminho mínimo.

Na Parte 1, a modelagem da malha aérea brasileira com 20 aeroportos e 73 arestas evidenciou como decisões metodológicas na definição de vértices, arestas e pesos impactam diretamente as propriedades estruturais do grafo. O modelo de pesos híbrido adotado — combinando penalidades por mudança de região

geográfica e ausência de hubs — mostrou-se adequado para refletir a realidade operacional da aviação civil: os algoritmos BFS e DFS confirmaram a conectividade do grafo, e o Dijkstra revelou que conexões via aeroportos hub reduzem significativamente o custo efetivo das rotas, com o par Manaus–São Paulo atingindo custo mínimo de 2,0 — o menor possível no modelo. A análise das métricas intra e inter-regionais confirmou uma relação inversa entre o número de aeroportos de uma região e sua densidade intra-regional, além de evidenciar o papel estrutural dos hubs como pontes entre comunidades pouco interligadas — resultado corroborado pelos valores de densidade de ego-rede (GRU: 0,409; BSB: 0,517; GIG: 0,564), os menores entre os 20 nós do grafo.

Na Parte 2, a modelagem do Universo Cinematográfico Marvel com 30 filmes e 56 arestas geradas programaticamente demonstrou a versatilidade da estrutura de grafos para domínios temáticos. A combinação dos critérios de categoria e ano de lançamento garantiu a conectividade do grafo — que, com apenas o critério de franquia, fragmentar-se-ia em 10 componentes desconexos —, viabilizando a aplicação de todos os quatro algoritmos em escala equivalente à da Parte 1. O Bellman-Ford foi aplicado com sucesso em dois cenários: convergindo corretamente com pesos negativos sem ciclo e detectando o ciclo negativo intencional (peso total -15,0) na verificação final, confirmando sua superioridade em relação ao Dijkstra para grafos com arestas de peso negativo. A comparação de desempenho entre os algoritmos foi consistente com as complexidades teóricas: BFS foi o mais rápido (média de 0,082 ms), seguido por DFS e Dijkstra (ambos em torno de 0,172 ms) e, por fim, Bellman-Ford (média de 0,689 ms), cuja maior lentidão decorre de sua complexidade $O(V \cdot E)$ e da ausência de parada antecipada no cenário com ciclo negativo.

A comparação entre os dois grafos também revelou diferenças estruturais significativas. A malha aérea apresentou distribuição de graus heterogênea (de 2 a 18, razão de 9:1), típica de redes hub-and-spoke, enquanto o grafo Marvel exibiu

distribuição homogênea (de 2 a 6, razão de 3:1), reflexo da geração de arestas por subgrafos completos de franquia. Essa diferença ressalta como a escolha dos critérios de modelagem determina não apenas a topologia do grafo, mas também o comportamento e a interpretabilidade dos resultados produzidos pelos algoritmos.

Do ponto de vista acadêmico, o projeto consolidou os conceitos teóricos abordados em sala de aula e os traduziu em soluções computacionais funcionais e bem documentadas. A implementação manual dos quatro algoritmos — sem o auxílio de bibliotecas de grafos externas como NetworkX —, a construção de uma interface interativa de visualização e a elaboração de testes unitários reforçaram tanto a compreensão teórica quanto as boas práticas de desenvolvimento de software. Como limitações, destacam-se o uso de pesos discretos na Parte 1 (em detrimento de distâncias geográficas reais) e a dependência de critérios relativamente grosseiros na Parte 2, que conectam filmes sem relação narrativa direta pelo simples fato de terem sido lançados no mesmo ano. Trabalhos futuros poderiam explorar pesos contínuos baseados em dados operacionais reais para a malha aérea, bem como critérios de conexão mais sofisticados para redes temáticas, como similaridade semântica ou redes de colaboração entre elencos.

REFERÊNCIAS

BRASIL. Ministério de Portos e Aeroportos. Belém (PA) concentra maior fluxo da aviação da Região Norte em janeiro. Brasília: Ministério de Portos e Aeroportos, 11 mar. 2026. Disponível em: <https://www.gov.br/portos-e-aeroportos/pt-br/assuntos/noticias/2026/03/belem-pa-concentra-maior-fluxo-da-aviacao-da-regiao-norte-em-janeiro>. Acesso em: 11 jun. 2026.

IBGE. Anuário Estatístico do Brasil 2023. Rio de Janeiro: IBGE, 2023. Disponível em: https://anuario.ibge.gov.br/images/aeb/2023/s1/2_pdf/s1t1103.pdf. Acesso em: 11 jun. 2026.

CESAR School | Cais do Apolo, 77, Bairro do Recife - Recife/PE CEP: 50030-390 - Fone: +55 (81) 3419.6700

MENEZES, Pedro. Ponte aérea lidera entre as rotas mais movimentadas do Brasil; veja ranking. Portal PANROTAS, São Paulo, 7 fev. 2025. Disponível em: https://www.panrotas.com.br/aviacao/pesquisas-e-estatisticas/2025/02/ponte-aerea-li-dera-entre-as-rotas-mais-movimentadas-do-brasil-em-2024-veja-ranking_214060.html. Acesso em: 11 jun. 2026.

TAFARELO, Saulo. Saiba quais são as rotas aéreas mais movimentadas do Brasil. CNN Brasil Viagem & Gastronomia, São Paulo, 14 fev. 2025. Disponível em: <https://www.cnnbrasil.com.br/viagemegastronomia/viagem/saiba-quais-sao-as-rotas-aereas-mais-movimentadas>